

ALGORİTMA VE PROGRAMLAMAYA GİRİŞ

8 ünite için sınav odaklı çalışma notları

BU DOSYADA

Temel kavramlar, karar yapıları, döngüler, fonksiyonlar, diziler, arama-sıralama algoritmaları ve bütünlük uygulamalar; adım adım çözülmüş sözde kod örnekleri; 40 soruluk genel deneme ve gerekçeli cevap anahtarı.

Nasıl çalışmalı?

- Her üniteye önce kalın yazılmış ayrımları öğren; sonra örnekte değişkenleri satır satır izle.
- Kodu okumakla yetinme: çıktıyı kapatıp aynı algoritmayı kendin elle çalıştır.
- Denemeyi bitirdikten sonra yanlışlarını ilgili ünitenin 'Sık hata' kısmıyla karşılaştır.

KAYNAK VE GÖSTERİM

Bu notlar, sağlanan **Algoritma ve Programlamaya Giriş** e-kitabının 8 ünitesi temel alınarak hazırlanmıştır. Kitap C#/Visual Basic ve çeşitli kaba kod gösterimlerini birlikte kullanır. Buradaki kodlar öğretici **sözde koddur**: köşeli parantezli diziler 0 tabanlı, **FOR i=a TO b** iki ucu dahil, = atama veya bağlama göre eşitlik anlamındadır. Gerçek dilde sözdizimi değişebilir.

ÜNİTE 01 / KİTAP PDF s. 3-23

Temel Kavramlar

BU ÜNİTEDE YAPABİLMELİSİN

Algoritmayı kurmak, akış şemasını okumak ve temel zaman karmaşıklığını ayırt etmek.

Temel bilgiler

- **Algoritma**, belirli bir girdiden istenen çıktıyı üretmek için sıralanmış ve sonlu adımlardır. Önce problem, girdi, beklenen çıktı ve özel durumlar tanımlanır; sonra çözüm küçük işlemlere ayrılır.
- İyi bir algoritmanın adımları **açık/kesin**, uygulanabilir/etkin, **sonlu** ve hedef için **doğru** olmalıdır. Her zaman en hızlı olması zorunlu değildir.
- **Girdi - işlem - çıktı**: Kullanıcıdan iki kenar uzunluğu al, çevre = $2 \times (\text{kısa} + \text{uzun})$ hesapla, çevreyi yazdır. Her algoritmada bu üç ögeyi aramak çözümü kolaylaştırır.
- Gösterim: doğal dil, **sözde kod (kaba kod)** ve **akış diyagramı**. Sözde kod bir programlama dilinin katı sözdizimine bağlı değildir; BAŞLA, OKU, IF, FOR, YAZ gibi ifadeler yeterlidir.

Akış diyagramı ve kontrol

- **Oval**: başla/bitiş; **paralelkenar**: girdi/çıktı; **dikdörtgen**: işlem; **eşkenar dörtgen**: koşul/karar; **ok**: akış yönü. Karar dalında doğru/yanlış yolları belirgin olmalıdır.
- Üç temel kontrol yapısı: **sıralı** (adımlar peş peşe), **seçimli** (koşula göre dal), **tekrarlı** (döngü). Bir çözüm bu yapıların birleşimidir.

Büyük O: sınavda tanı

- **Büyük O**, n girdi büyürken işlem/bellek ihtiyacının yaklaşık büyümesini anlatır. Sabitleri ve küçük dereceli terimleri önemseme: $3n + 7 \rightarrow O(n)$; $n^2 + n \rightarrow O(n^2)$.
- Diziye doğrudan erişim $O(1)$; tüm elemanları bir kez gezme $O(n)$; her adımda aralığı ikiye indirme $O(\log n)$; iki tam iç içe döngü $O(n^2)$. Aynı düzeyde ardışık iki döngü $O(n) + O(n) = O(n)$.
- En iyi/ortalama/en kötü durum farklı olabilir. Karmaşıklık **doğruluk** ile aynı şey değildir; hızlı ama yanlış algoritma çözüm sayılmaz.

Çözümlü örnek: 1'den n'e toplam

```
BAŞLA
OKU n
toplam = 0
FOR i = 1 TO n
    toplam = toplam + i
NEXT i
YAZ toplam
BİTİR
```

n = 4 için toplam sırasıyla 1, 3, 6, 10 olur; çıktı 10'dur. Döngü n kez çalıştığı için zaman $O(n)$, tutulan değişken sayısı sabit olduğu için ek alan $O(1)$.

SIK HATA / SINAV İPUCU

Akış şemasındaki karar sembolünü işlem sembolüyle karıştırma. n=0 gibi sınır girdileri ve algoritmanın gerçekten bitip bitmediğini kontrol et.

ÜNİTE 01 / UYGULAMA

Algoritmayı ve karmaşıklığı izle

Bir soruda önce girdi/çıktıyı, sonra tekrar sayısını ve durma koşulunu bul. İşlem adımlarını tabloya dökmek özellikle karmaşıklık sorularında işe yarar.

Örnek 1 - Karar + işlem

Girdi $a=7$, $b=4$ için aşağıdaki algoritma büyük değeri yazar:

```
OKU a, b
IF a > b THEN büyük = a
ELSE büyük = b
YAZ büyük
```

$7 > 4$ doğru olduğundan $büyük=7$ ve çıktı 7. Koşul bir kez sınanır; zaman ve ek alan $O(1)$.

Örnek 2 - Ardışık döngüler

İki döngü iç içe değilse süreler toplanır:

```
FOR i = 1 TO n: YAZ i
FOR j = 1 TO n: YAZ j
```

Toplam $2n$ kez yazılır, Büyük O'da sabit çarpan atılır: $O(n)$. İkinci FOR birincinin içinde olsaydı $n \times n = O(n^2)$.

Hızlı kontrol

Bir algoritma mutlaka en hızlı çözüm müdür? Hayır; doğruluk ve sonluluk gereklidir, en hızlı olmak şart değildir.

Girdi büyürken işlem sayısı hiç artmıyorsa? $O(1)$.

İşlem sayısı $5n^2+3n+2$ ise? $O(n^2)$; baskın terim n^2 'dir.

ÜNİTE 02 / KİTAP PDF s. 24-48

Programlamaya Giriş

BU ÜNİTEDE YAPABİLMELİSİN

Veri tipi, değişken, sabit ve operatörlerle sıralı işlemleri çözmek.

Veri tiplerini seç

- **Tam sayı:** byte/short/int/long gibi türler; işaretli türler negatif ve pozitif, işaretsiz türler sıfır ve pozitif değer alır.
Ondalıklı: single/float, double ve decimal. Yaklaşık ölçümler için kayan nokta, para gibi kesin ondalık gereksinimi için decimal düşün.
- **char** tek karakter; **string** birden fazla karakterli metin; **boolean** doğru/yanlış; **date** tarih-zaman bilgisidir. '7' karakteri ile 7 sayısı aynı veri değildir.
- **Değişken** adı, tipi ve çalışma sırasında değişebilen değeri olan saklama yeridir. **Sabit** bir kez değer alır, sonra değiştirilmez. Anlamlı ad seç; sayı ile başlayan ad kullanma; dilin ayrılmış sözcüklerini ad olarak verme.

Operatörler ve işlem sırası

- Aritmetik: +, -, ×, /, mod (%). **17 mod 5 = 2** çünkü $17 = 3 \times 5 + 2$. Tam sayı bölmesi ve normal bölmenin sonucu dilin kurallarına göre değişebilir.
- Karşılaştırmalar (<, >, <=, >=, eşit, eşit değil) boolean sonuç verir. Atama **x = x + 1** matematiksel eşitlik değildir: sağdaki eski x hesaplanır, soldaki x'e yeni değer yazılır.
- **AND:** her iki koşul doğruysa doğru. **OR:** en az biri doğruysa doğru. **NOT:** sonucu tersine çevirir. Parantez karmaşık koşullarda yanılmayı azaltır.
- Sıralı akışta hiçbir dal veya döngü yoktur; satırlar yukarıdan aşağıya birer kez çalışır. İzleme sorusunda her atamadan sonra değişkenleri güncelle.

Çözümlü örnek: değişken izle

```
x = 5
y = 2
x = x + y      // x = 7
y = x mod 3    // y = 1
koşul = (x > 6) AND (y = 1)  // Doğru
```

Son durum x=7, y=1, koşul=Doğru. Önce sağ taraf hesaplanır, sonra atama yapılır. Gerçek bir dilde karşılaştırma işareti = veya == olabilir; bu notlarda sözde kod kullanılır.

SIK HATA / SINAV İPUCU

Veri tipi aralıkları dile göre değişebilir. Özellikle / ve tam sayı bölmesini, ayrıca karakter ile sayıyı ayır.

ÜNİTE 02 / UYGULAMA

Veri tipini seç ve ifadeyi hesapla

Veri tipini beklenen değer ve yapılacak işleme göre seç. Kimlik/telefon gibi başında sıfır bulunabilen ve aritmetik işlem gerektirmeyen veriler çoğu uygulamada metin olarak tutulur.

Örnek 1 - Tip seçimi

Aşağıdaki değerler için en doğal türleri eşleştir:

```
öğrenci_sayısı = 120      -> tam sayı
birim_fiyat = 19,95      -> kesin ondalık
harf_notu = 'A'          -> char
ad_soyad = 'Ayşe Yılmaz' -> string
mezun_mu = Doğru        -> boolean
```

Sayının ondalık olması tek başına double gerektirmez; parasal değerlerde decimal gibi kesin ondalık türü daha uygun olabilir.

Örnek 2 - İşlem izi

Atama sırasını değiştirince sonuç da değişir:

```
a = 8; b = 3
a = a - b      // a=5, b=3
b = a + b      // a=5, b=8
YAZ a, b      // 5, 8
```

İkinci satırdaki a artık 8 değil 5'tir. Her satır sonunda değer tablosunu yenile.

Hızlı kontrol

'5' ile 5 aynı mı? Hayır; ilki karakter/metin, ikincisi sayısal değerdir.

(4>2) AND (3<1) sonucu? Yanlış; ikinci koşul yanlıştır.

(4>2) OR (3<1) sonucu? Doğru; en az bir koşul doğrudur.

ÜNİTE 03 / KİTAP PDF s. 49-79

Karar Yapıları

BU ÜNİTEDE YAPABİLMELİSİN

Tek, çift, çok seçimli ve iç içe koşulları izlemek ve yazmak.

Koşul türleri

- **Tek seçim (IF):** koşul doğruysa blok çalışır, yanlışsa blok atlanır. **Çift seçim (IF/ELSE):** iki yoldan yalnız biri çalışır.
- **Çok seçim (IF/ELSEIF/ELSE):** koşullar sırayla denir, ilk doğru dal seçilir. **İç içe IF:** bir kararın dalında yeni karar verilir. Girinti hangi ELSE'in hangi IF'e ait olduğunu gösterir.
- Sınır değerlerinde > ile >= farklı sonuç verir. '50 ve üzeri geçti' için not >= 50 yazılır. Aralık koşulunda örneğin 50 <= not AND not < 70 kullanılır.
- Çoklu dallarda eşikler yüksekten düşüğe sıralanırsa bazı kontroller sadeleşir: not >= 90, sonra >= 70, sonra >= 50. ELSE kalan tüm durumları kapsar.

Kısa izleme yöntemi

- Girdiyi her koşula sırayla yerleştir; doğru/yanlış işaretle; ilk doğru dalı bul; yalnız o dalın atamalarını yap. İç içe yapıda önce dış koşulu, sonra seçilen dalın iç koşulunu değerlendir.
- **AND** aralığı daraltır, **OR** seçenekleri birleştirir. Örneğin '18-65 yaş dahil' için yaş >= 18 AND yaş <= 65.

Çözümlü örnek: not sınıflandırması

```
OKU not
IF not >= 90 THEN
    harf = "AA"
ELSEIF not >= 70 THEN
    harf = "BB"
ELSEIF not >= 50 THEN
    harf = "CC"
ELSE
    harf = "FF"
END IF
YAZ harf
```

not=70 için ilk koşul yanlış, ikinci doğru; sonuç BB. not=49 için tüm koşullar yanlış; sonuç FF. Bir kez dal seçildikten sonra sonraki dallar denenmez.

SIK HATA / SINAV İPUCU

ELSEIF sırası önemlidir. '>=50' kontrolü '>=90'dan önce olsaydı 95 için de yanlışlıkla CC seçilirdi.

ÜNİTE 03 / UYGULAMA

Karar ağacını elle çalıştır

Koşullarda sınır değerlerini özellikle dene. 49, 50, 69, 70 ve 90 gibi sayılar yanlış $\geq$ veya yanlış ELSEIF sırasını hemen ortaya çıkarır.

Örnek 1 - Eşik izleme

Ünitedeki not sınıflandırmasını şu dört girdiyle izle:

```
not=49 -> >=90 Y, >=70 Y, >=50 Y -> FF
not=50 -> >=90 Y, >=70 Y, >=50 D -> CC
not=70 -> >=90 Y, >=70 D -> BB
not=90 -> >=90 D -> AA
```

Burada D=Doğru, Y=Yanlış. İlk doğru koşuldan sonra kalan ELSEIF dalları atlanır.

Örnek 2 - İç içe koşul

Ücretsiz teslimat, üyeliğe ve sepet tutarına birlikte bağlı olsun:

```
IF üye_mi THEN
  IF tutar >= 200 THEN YAZ 'Ücretsiz'
  ELSE YAZ 'Ücretli'
ELSE YAZ 'Ücretli'
```

üye_mi=Yanlış ise iç koşul hiç sınanmaz. üye_mi=Doğru, tutar=200 ise 'Ücretsiz' yazılır.

Hızlı kontrol

95 için ≥ 50 kontrolü en önceyse hangi dal seçilir? İlk dal; bu yüzden eşikler doğru sırada olmalı.

yaş=65, yaş $\leq$ 65 doğru mu? Evet; sınır dahildir.

IF koşulu yanlış, ELSE yoksa? Blok atlanır, sonraki satırdan devam edilir.

ÜNİTE 04 / KİTAP PDF s. 80-103

Döngüler

BU ÜNİTEDE YAPABİLMELİSİN

Tekrar sayısı belli/belirsiz döngüleri, sayaç ve toplamı doğru izlemek.

Döngü seçimi

- **WHILE/DO WHILE:** koşul başta sınanır, blok sıfır kez çalışabilir. **DO ... LOOP WHILE:** koşul sonda sınanır, blok en az bir kez çalışır. Kitaptaki yazım VB tarzı olabilir; temel ayırım kontrolün yeri ve ilk çalışmadır.
- **FOR:** tekrar sayısı veya sayaç aralığı biliniyorsa uygundur. Bu notlarda 'FOR i=1 TO 5' uçlar dahil 5 tekrar demektir. **FOR EACH:** koleksiyonun elemanlarını sırayla gezer.
- Döngüde **başlangıç, koşul, güncelleme** üçlüsünü kontrol et. Güncelleme yoksa koşul hiç değişmeyebilir ve sonsuz döngü oluşur. Koşul tersse döngü hiç başlamayabilir.

Sayaç, birikim ve sınır

- **Sayaç** adet sayar: $adet = adet + 1$; ilk değeri 0. **Toplam** biriktirir: $toplam = toplam + sayi$; ilk değeri 0. **Çarpım** biriktirir: $çarpım = çarpım \times sayi$; ilk değeri 1.
- Sentinel (bitirme değeri) kullanıldığında o değeri toplama/ortalama katma. Ortalama = $toplam / adet$ hesaplamadan önce $adet > 0$ denetle.
- Bir tam dizi taraması $O(n)$; her eleman için tüm elemanları gezen iç içe iki döngü $O(n^2)$. İç döngü sayısı sabitse karmaşıklık farklı olabilir; kodu gerçek tekrar sayısıyla oku.

Çözümlü örnek: -1 girilene dek ortalama

```
toplam = 0; adet = 0
OKU sayı
WHILE sayı != -1
    toplam = toplam + sayı
    adet = adet + 1
    OKU sayı
END WHILE
IF adet > 0 THEN YAZ toplam / adet
ELSE YAZ "Sayı girilmedi"
```

Girdiler 4, 8, -1 ise toplam=12, adet=2, sonuç=6. -1 hesaplamaya girmez. İlk girdi -1 ise döngü sıfır kez çalışır ve sıfıra bölme önlenir.

SIK HATA / SINAV İPUCU

FOR i=0 TO n-1 toplam n kez döner; FOR i=0 TO n ise n+1 kez döner. Çıkış koşulunu ve son elemanın dahil olup olmadığını işaretle.

ÜNİTE 04 / UYGULAMA

Döngünün kaç kez çalıştığını bul

Döngü sorularında üç sütun çiz: sayaç/okunan değer, koşulun sonucu, biriken sonuç. Çıkış kontrolünün başta mı sonda mı olduğunu unutma.

Örnek 1 - Çift sayıların toplamı

1'den 6'ya kadar yalnız çift sayıları topla:

```
toplam = 0
FOR i = 1 TO 6
    IF i mod 2 = 0 THEN toplam = toplam + i
NEXT i
YAZ toplam      // 2+4+6 = 12
```

FOR altı kez çalışır; IF yalnız i=2,4,6 için toplama yapar. Bir döngünün içinde IF olması karmaşıklığı $O(n)$ 'den $O(n^2)$ 'ye çıkarmaz.

Örnek 2 - Başta/sonda kontrol

koşul başlangıçta Yanlış olsun:

```
WHILE koşul: YAZ 'A'      // 0 kez
DO: YAZ 'B': LOOP WHILE koşul // 1 kez
```

Bu iki döngünün farkı ilk çalışmanın garanti olup olmamasıdır. Sonraki tekrarlar koşulun güncellenmesine bağlıdır.

Hızlı kontrol

FOR i=0 TO 4 kaç tekrar? 5 tekrar.

Çarpım biriktirme başlangıcı kaç? 1; 0 seçilirse sonuç hep 0 olur.

Sentinel -1 ortalamaya dahil mi? Hayır; yalnız bitirmeyi bildirir.

ÜNİTE 05 / KİTAP PDF s. 104-130

Alt Programlar ve Fonksiyonlar

BU ÜNİTEDE YAPABİLMELİSİN

Parametre, dönüş değeri, ana program ve özyinelemeyi ayırt etmek.

Alt program mantığı

- **Alt program** bir görevi kapsayan, gerektiğinde çağrılan kod parçasıdır. Kod tekrarını azaltır, hata ayıklamayı ve bakımı kolaylaştırır. Ana program (main) işlemleri ve çağrıları başlatır.
- **Parametre** tanımda yazılan girdi adıdır; **argüman** çağrı sırasında gönderilen gerçek değerdir. `alan(en, boy)` tanımında en/boy parametre, `alan(3, 4)` çağrısında 3/4 argümandır.
- **Void/prosedür** çağırana sonuç değeri döndürmez; yazdırma veya güncelleme yapabilir. **Fonksiyon** RETURN ile değer döndürür, bu değer değişkene atanabilir veya ifadede kullanılabilir.
- Yerel değişken yalnız tanımlandığı alt programda geçerlidir. Parametreye verilen değer dışarıyı değiştirip değiştirmemesi dilin değer/referans aktarım kurallarına bağlıdır; sınavda kodun verdiği kurala bak.

Özyineleme (recursion)

- Özyinelemeli fonksiyon kendisini daha küçük bir problemle çağırır. **Temel durum** durmayı sağlar; **özyinelemeli adım** problemi küçültür. Temel durum yoksa bitmeyen çağrı oluşabilir.
- Faktöriyel: $0! = 1$; $n! = n \times (n-1)!$ ($n > 0$). Örneğin $F(3) = 3 \times F(2) = 3 \times 2 \times F(1) = 6$.

Çözümlü örnek: alan fonksiyonu ve çağrı

```
FONKSİYON Alan(en, boy)
    RETURN en * boy
SON

ANA PROGRAM
    sonuç = Alan(3, 4)
    YAZ sonuç      // 12
SON
```

Alan iki parametre alır ve bir sayı döndürür. Ana programdaki sonuç 12 olur. Yalnız ekrana yazdıran bir alt program olsaydı dönüş değeri atanmazdı.

SIK HATA / SINAV İPUCU

RETURN ile YAZ farklıdır: RETURN çağırana değer verir, YAZ ekrana çıktı üretir. Fonksiyon çağrısını tanımla karıştıрма.

ÜNİTE 05 / UYGULAMA

Fonksiyon çağrısını ve dönüşünü izle

Fonksiyon sorusunda çağrı anındaki argümanları parametrelere yerleştir, RETURN satırını bul, dönen değeri çağrının yerine yaz.

Örnek 1 - İki çağrı

Aynı fonksiyon farklı argümanlarla yeniden kullanılabilir:

```
FONKSİYON Kare(x): RETURN x*x
a = Kare(3)           // 9
b = Kare(a-5)         // Kare(4) -> 16
YAZ a+b               // 25
```

İkinci çağrıda argüman $a-5=4$ 'tür. Fonksiyon tanımı bir kere yazılır, her çağrıda yeni girdiyle çalışır.

Örnek 2 - Özyineleme

$F(3)$ çağrısının açılıp kapanışını izle:

```
F(0) = 1
F(n) = n * F(n-1)
F(3) -> 3*F(2) -> 3*2*F(1)
      -> 3*2*1*F(0) -> 6
```

Her çağrı n'yi azaltır; $F(0)$ temel durumdur. $F(0)$ tanımsız olsaydı çağrılar durmazdı.

Hızlı kontrol

Fonksiyonun döndürdüğü değer nasıl alınır? Örneğin sonuç = $Kare(3)$ ile.

YAZ ile RETURN aynı mı? Hayır; YAZ çıktı üretir, RETURN çağrıya değer verir.

Argüman nerede görünür? Çağrıdaki gerçek değer veya ifade olarak.

ÜNİTE 06 / KİTAP PDF s. 131-154

Diziler

BU ÜNİTEDE YAPABİLMELİSİN

Tek/iki boyutlu dizilerde indisleri ve döngü aralıklarını doğru kullanmak.

Tek boyutlu diziler

- **Dizi**, aynı türdeki değerleri tek ad altında sıralı tutar. Her elemana **indis** ile erişilir. Bu notlarda indis 0'dan başlar: n elemanlı dizinin geçerli indisleri 0 ... n-1'dir.
- Örnek D = [12, 7, 20, 5] için D[0]=12, D[2]=20, son eleman D[3]=5. D[4] sınır dışıdır. Kitap bazı örneklerde 1 tabanlı gösterim de kullanır; soruda belirtilen başlangıç indisini esas al.
- Toplam/ortalama için tüm elemanları bir kez dolaş; min/max için ilk elemanla başlatıp kalanları karşılaştır. Boş dizi için min/max tanımsız, ortalama için sıfıra bölme riskidir.

İki boyutlu diziler

- **Matris** satır ve sütunla belirtilir: M[satır][sütun]. 2 × 3 matrisin 2 satırı ve 3 sütunu, toplam 6 elemanı vardır. Satır için dış, sütun için iç döngü sık kullanılır.
- Tüm r × c elemanları gezmenin zamanı O(r × c)'dir. Yalnız r=c=n ise O(n²) yazılır. Tek bir indisten elemana doğrudan erişim O(1)'dir.

Çözümlü örnek: dizinin en küçüğü

```
D = [12, 7, 20, 5]
en_küçük = D[0]
FOR i = 1 TO 3
    IF D[i] < en_küçük THEN en_küçük = D[i]
NEXT i
YAZ en_küçük      // 5
```

İzleme: ilk değer 12; i=1'de 7; i=2'de yine 7; i=3'te 5. Tüm elemanlar en fazla bir kez karşılaştırıldığı için O(n).

SIK HATA / SINAV İPUCU

Dizi uzunluğu ile son indis aynı değildir. İki boyutlu dizide ilk indis satır, ikinci indis sütundur (aksi özellikle belirtilmedikçe).

ÜNİTE 06 / UYGULAMA

İndisleri ve matrisi izle

İndis başlangıcı verilmemişse kitabın ilgili örneğine bak; bu notların tüm uygulamaları 0 tabanlıdır.

Örnek 1 - Koşulu sağlayanları say

$D=[4,7,2,9,6]$ için 5'ten büyük eleman sayısı:

```
adet = 0
FOR i = 0 TO 4
    IF D[i] > 5 THEN adet = adet + 1
NEXT i
YAZ adet          // 3
```

Koşulu 7, 9, 6 sağlar. Beş eleman gezilir; $i=5$ geçersiz olurdu.

Örnek 2 - Matris toplamı

$M=[[1,2,3],[4,5,6]]$ matrisinin tüm elemanlarını topla:

```
toplam = 0
FOR satır = 0 TO 1
    FOR sütun = 0 TO 2
        toplam = toplam + M[satır][sütun]
YAZ toplam          // 21
```

2 satır $\times$ 3 sütun = 6 eleman. $M[1][0]=4$; $M[0][2]=3$. Dış/ iç döngü sınırları matris boyutlarına göre ayrı belirlenir.

Hızlı kontrol

6 elemanlı 0 tabanlı dizinin son indisi? 5.

$M[1][2]$ hangi konum? İkinci satır, üçüncü sütun.

$r \times c$ matrisin tümünü gezme süresi? $O(r \times c)$.

ÜNİTE 07 / KİTAP PDF s. 155-181

Arama ve Sıralama Algoritmaları

BU ÜNİTEDE YAPABİLMELİSİN

Temel algoritmaların çalışma biçimini, ön şartını ve karmaşıklığını karşılaştırmak.

Arama

- **Doğrusal arama:** baştan sona tek tek karşılaştırır; sıralı dizi gerekmez. İlk elemanda bulursa en iyi $O(1)$, bulunamazsa veya sonda ise en kötü $O(n)$.
- **İkili arama:** dizi **önceden sıralı** olmalıdır. Ortayı karşılaştır; aranan küçükse sol yarıya, büyükse sağ yarıya geç. Her adımda aralık yarılanır; en kötü $O(\log n)$. Sıralama maliyeti ayrıca değerlendirilir.

Sıralama

- **Kabarcık:** komşu elemanları kıyasla, yanlış sıradaysa yer değiştir; büyük elemanlar sona ilerler. Tipik/en kötü $O(n^2)$.
- **Yerleştirmeli:** solda sıralı bölüm oluştur; yeni elemanı uygun yere kaydırarak yerleştir. Sıralıya yakın dizide iyi çalışır; en iyi $O(n)$, en kötü $O(n^2)$.
- **Hızlı sıralama:** pivot seç, küçük/büyük gruplara ayır, grupları yinele. Ortalama $O(n \log n)$, kötü pivotlarla en kötü $O(n^2)$.
- **Birleştirmeli:** tek elemana kadar böl; sıralı parçaları birleştir. Her durumda $O(n \log n)$, ek bellek çoğu uygulamada $O(n)$.

Çözümlü örnek: ikili arama izi

```
D = [3, 7, 11, 18, 24, 31, 40]; aranan = 24
sol=0, sağ=6 -> orta=3 -> D[3]=18 < 24
sol=4, sağ=6 -> orta=5 -> D[5]=31 > 24
sol=4, sağ=4 -> orta=4 -> D[4]=24, bulundu
```

Üç karşılaştırma yapıldı. Dizinin sıralı oluşu sayesinde her adımda diğer yarı elendi. Sırasız dizide aynı mantık güvenilir değildir.

SIK HATA / SINAV İPUCU

Kitap hızlı sıralama için $O(n \log n)$ verir; bu ortalama/beklenen durumdur, en kötü durum $O(n^2)$ 'dir. İkili aramanın sıralı dizi şartını mutlaka yaz.

ÜNİTE 07 / UYGULAMA

Arama ve sıralama adımlarını karşılaştır

Algoritmanın adını ezberlemek yerine hangi elemanları hangi sırayla karşılaştırdığına bak. Özellikle sıralı dizi ön şartını ve pivotun rolünü ayırt et.

Örnek 1 - Kabarcıkta ilk tur

$D=[8,3,6,1]$ için komşuları artan sırada karşılaştır:

```
[8,3,6,1] -> [3,8,6,1]
           -> [3,6,8,1]
           -> [3,6,1,8]
```

İlk turun sonunda en büyük değer 8 sona yerleşir. Dizi henüz sıralı değildir; sonraki turlarda 3,6,1 düzenlenir.

Örnek 2 - Birleştirmeli

$D=[9,4,7,2]$ önce bölünür, sonra sıralı parçalar birleşir:

```
[9,4,7,2] -> [9,4] [7,2]
           -> [9] [4] [7] [2]
           -> [4,9] [2,7]
           -> [2,4,7,9]
```

Hızlı sıralamada pivotla bölümleme; birleştirmeli sıralamada ayrı sıralı parçaları bir araya getirme vurgulanır.

Hızlı kontrol

Sırasız dizide doğrudan ikili arama güvenilir mi? Hayır; önce sıralama gerekir.

Hızlı sıralamanın en kötü süresi? $O(n^2)$; kötü pivot seçimi olabilir.

Yerleştirmeli sıralama sıralıya yakın dizide? En iyi durumda $O(n)$ 'e yaklaşır.

ÜNİTE 08 / KİTAP PDF s. 182-223

Genel Uygulamalar

BU ÜNİTEDE YAPABİLMELİSİN

Girdi, karar, döngü, dizi ve fonksiyonu birlikte kullanarak problemleri çözmek.

Problemi çözme şablonu

- 1) Girdileri ve çıktı biçimini yaz. 2) Özel durumları belirle (boş dizi, sıfır, negatif değer, sınır notu). 3) Gerekli değişkenleri uygun ilk değerle başlat. 4) Koşul ve döngüyü kur. 5) Küçük örnekle elle izle. 6) Sonucu yazdır.
- Sayma:** sayaç=0, koşulu sağlayan her olayda sayaç++. **Toplama:** toplam=0, her elemanda ekle. **Çarpma:** sonuç=1, her adımda çarp. **En küçük/en büyük:** ilk veriyle başlatıp karşılaştır.
- Ağırlıklı ortalama:** örneğin vize %40 ve final %60 ise ortalama = $0,40 \times \text{vize} + 0,60 \times \text{final}$. Yüzdelerin toplamı %100 olmalı. Geçme koşulu soruda veriliyorsa onu kullan.

Birleşik örneklerde sınav yöntemi

- Dizi + koşul + döngü sorusunda her indis için koşulun doğru olup olmadığını tabloya yaz; sayaç ve toplamı satır satır güncelle. 20°C altı ve 20°C ve üstü gibi eşikler birbirini dışlamalı.
- Basamak saymada sayı=0 özel durumdur (1 basamak). Pozitif sayıda tekrar tekrar 10'a tam böl; kaç kez bölündüğünü say. Negatif sayı için mutlak değeri al.
- Sentinel ile okuma, eksik veri, sıfıra bölme ve dizi sınırı en sık hata kaynaklarıdır. Son yazdırılan şeyin hesaplanan değişken olduğundan emin ol.

Çözümlü örnek: sıcak gün sayısı

```
sıcak_gün = 0
FOR i = 0 TO n-1
    IF sıcaklık[i] >= 20 THEN
        sıcak_gün = sıcak_gün + 1
    END IF
NEXT i
YAZ sıcak_gün
```

sıcaklık=[18, 20, 25, 19] ise koşul sırasıyla yanlış, doğru, doğru, yanlış; çıktı 2. Sınır değer 20 dahil edildi. Dizi bir kez gezildiği için O(n).

SIK HATA / SINAV İPUCU

Birden çok koşulda sınır değerleri boşta bırakma ya da iki dala birden sokma. Ortalama hesaplamadan önce adet=0 durumunu denetle.

ÜNİTE 08 / UYGULAMA

Birleşik problemleri parçala

Uygulama sorularında formül + koşul + döngü birlikte gelir. Önce veri doğrulama ve boş veri durumunu, sonra asıl hesaplamayı kur.

Örnek 1 - Ağırlıklı not ve karar

Vize/final yüzdeleri soruda %40/%60 verilmiş olsun:

```
OKU vize, final
ortalama = 0.4*vize + 0.6*final
IF ortalama >= 50 THEN YAZ 'Geçti'
ELSE YAZ 'Kaldı'
```

vize=70, final=80 için ortalama=76 ve sonuç 'Geçti'. Gerçek dersin geçme kuralları farklı verildiyse sorudaki koşul esas alınır.

Örnek 2 - Basamak sayısı

Sıfır ve negatif girdi için de çalışan temel fikir:

```
OKU sayı
x = mutlak(sayı); basamak = 1
WHILE x >= 10
    x = x tam_böl 10
    basamak = basamak + 1
YAZ basamak
```

0 için döngüye girilmez, çıktı 1'dir. -125 için mutlak değer 125 alınır; 125→12→1 olduğundan çıktı 3'tür.

Hızlı kontrol

5 veri için ortalamada bölen kaçtır? 5; geçerli veri sayısı kullanılır.

20°C eşikte 'sıcak' >=20 ise 20 nereye gider? Sıcak gün grubuna.

Bir uygulama algoritmasını nasıl kontrol edersin? Normal, sınır ve boş/özel girdilerle elle izle.

SON TEKRAR

Bir bakışta ayrımlar

Konu	Hatırlanacak kural
Algoritma	Açık, uygulanabilir, sonlu ve doğru adımlar.
IF / ELSE	IF tek seçim; IF/ELSE iki daldan biri; ELSEIF ilk doğru koşul.
WHILE / DO...LOOP	Başta koşul: 0 tekrar mümkün. Sonda koşul: en az 1 tekrar.
FOR	Tekrar aralığı belliyse; bu notlarda TO uçları dahil.
Fonksiyon / prosedür	Fonksiyon değer döndürür; void/prosedür döndürmez.
Dizi	0 tabanlı n elemanda son indis n-1; matriste satır ve sütun.
İkili arama	Sıralı dizi şart; $O(\log n)$.
Sıralama	Kabarcık/yerleştirmeli en kötü $O(n^2)$; birleştirmeli $O(n \log n)$.

Yapı	Tipik süre	Neden
Doğrudan erişim	$O(1)$	Adım sayısı n'den bağımsız.
İkili arama	$O(\log n)$	Aralık her adımda yarılanır.
Tek tam tarama	$O(n)$	n elemanın her biri incelenir.
İç içe iki tam tarama	$O(n^2)$	$n \times n$ tekrar yapılır.

DENEMEDEN ÖNCE

Sözde kodda **önce sağ tarafı hesapla, sonra sol tarafa ata**. Koşulları sırayla izle. Döngüde kaç tekrar olduğunu ve son sınırın dahil olup olmadığını yaz. Bir matriste önce satırı, sonra sütunu bul.

GENEL DENEME

40 özgün soru

Her üniteden 5 soru vardır. Önce cevapları kapatarak çöz; gerekçeleri sondaki anahtardan kontrol et.

Ünite 1

1. Bir algoritmanın sonlu sayıda adım sonunda durması hangi özelliktir?
A) Kesinlik B) Sonluluk C) Görsellik D) En hızlı olma
2. Akış diyagramında bir koşul hangi şekille gösterilir?
A) Oval B) Dikdörtgen C) Eşkenar dörtgen D) Paralelkenar
3. Her biri n kez çalışan iki iç içe döngünün zamanı genellikle nedir?
A) $O(1)$ B) $O(\log n)$ C) $O(n)$ D) $O(n^2)$
4. Programlama dilinden bağımsız adım anlatımı hangisidir?
A) Makine kodu B) Sözde kod C) İkili sayı D) Derleyici
5. Bir dizinin belirli indisine doğrudan erişim genellikle hangi karmaşıklıktadır?
A) $O(1)$ B) $O(n)$ C) $O(n^2)$ D) $O(2^n)$

Ünite 2

6. Birden fazla karakterden oluşan metin için hangi tip uygundur?
A) char B) boolean C) string D) int
7. Kesirli ölçüm saklamak için hangisi daha uygundur?
A) int B) double C) boolean D) char
8. $17 \bmod 5$ işleminin sonucu kaçtır?
A) 1 B) 2 C) 3 D) 5
9. $A \text{ AND } B$ hangi durumda doğrudur?
A) Yalnız A doğruysa B) Yalnız B doğruysa C) En az biri doğruysa D) İkisi de doğruysa
10. $x=4$ iken $x=x+1$ sonrası x kaç olur?
A) 4 B) 5 C) 1 D) Hata

Ünite 3

11. Yalnız IF bulunan yapıda koşul yanlışsa ne olur?
A) Blok atlanır B) Blok iki kez çalışır C) ELSE çalışır D) Program mutlaka hata verir
12. IF/ELSE yapısında tek çalışmada kaç dal yürütülür?
A) Sıfır B) Bir C) İki D) Sonsuz
13. $\text{not}=95$ için önce $\text{not}\geq 50$ sonra ELSEIF $\text{not}\geq 90$ sınıırılsa ilk dal hangisidir?
A) İkinci dal B) Hiçbiri C) İlk dal D) İki dal
14. '18-65 yaş dahil' koşulu hangisidir?
A) $\text{yaş}\geq 18 \text{ OR } \text{yaş}\leq 65$ B) $\text{yaş}\geq 18 \text{ AND } \text{yaş}\leq 65$ C) $\text{yaş}=18 \text{ AND } \text{yaş}=65$ D) $\text{yaş}\leq 18 \text{ OR } \text{yaş}\geq 65$
15. İç içe IF ne demektir?
A) IF'in bir dalında başka IF bulunması B) Döngünün koşulsuz olması C) Her dalın çalışması D) Aynı satırın tekrarı

Ünite 4

16. WHILE döngüsü koşulu başlangıçta yanlışsa kaç kez çalışabilir?
A) Sıfır B) Bir C) İki D) Sonsuz
17. Koşulu sonda denetlenen DO...LOOP WHILE en az kaç kez çalışır?
A) Sıfır B) Bir C) İki D) n kez
18. Uçlar dahil FOR i=1 TO 5 kaç kez çalışır?
A) 4 B) 5 C) 6 D) Belirsiz
19. Bir toplam değişkeninin güvenli başlangıcı hangisidir?
A) -1 B) 0 C) 1 D) n
20. n kez dış ve n kez iç döngü çalışırsa toplam tekrar yaklaşık kaçtır?
A) 1 B) $\log n$ C) n D) n^2

Ünite 5

21. Void alt programın ayırt edici özelliği nedir?
A) Parametre alamaz B) Değer döndürmez C) Yazdıramaz D) Çağrılmaz
22. FONKSİYON Alan(en,boy) tanımındaki en ve boy nedir?
A) Argüman B) Sabit C) Parametre D) Dizi
23. Bir fonksiyonun hesapladığı değeri çağırana ileten ifade hangisidir?
A) OKU B) YAZ C) FOR D) RETURN
24. Faktöriyel özyinelemesinde doğru temel durum hangisidir?
A) $F(0)=1$ B) $F(0)=0$ C) $F(n)=n+n$ D) $F(1)=0$
25. Özyinelemede temel durum yoksa hangi risk doğar?
A) Dizi sıralanır B) Bitmeyen çağrı/taşma C) Veri tipi değişir D) Her zaman $O(1)$ olur

Ünite 6

26. 0 tabanlı n elemanlı dizinin son indisi nedir?
A) n B) $n+1$ C) $n-1$ D) 1
27. $D=[8,3,6,9]$ için $D[2]$ kaçtır?
A) 3 B) 6 C) 8 D) 9
28. Temel dizi tanımında elemanların veri tipleri nasıldır?
A) Aynı tip B) Her zaman farklı C) Yalnız string D) Yalnız sayı
29. $M=[[1,2,3], [4,5,6]]$ için $M[1][2]$ kaçtır?
A) 2 B) 3 C) 5 D) 6
30. r satır ve c sütunlu matrisin tümünü gezme zamanı nedir?
A) $O(1)$ B) $O(r+c)$ C) $O(r \times c)$ D) $O(\log r)$

Ünite 7

31. İkili aramanın temel ön şartı nedir?

A) Dizi sıralı olmalı B) Dizi boş olmalı C) İki döngü olmalı D) Değer mutlaka bulunmalı

32. Doğrusal aramanın en kötü zaman karmaşıklığı nedir?

A) $O(1)$ B) $O(\log n)$ C) $O(n)$ D) $O(n^2)$

33. Komşu öğeleri karşılaştırıp yer değiştiren yöntem hangisidir?

A) Birleştirmeli B) Kabarcık C) İkili arama D) Hızlı

34. Pivot seçen sıralama yöntemi hangisidir?

A) Yerleştirmeli B) Kabarcık C) Birleştirmeli D) Hızlı

35. Birleştirmeli sıralamanın genel zaman karmaşıklığı nedir?

A) $O(1)$ B) $O(n)$ C) $O(n \log n)$ D) $O(n^2)$

Ünite 8

36. Vize 70 (%40), final 80 (%60) ise ağırlıklı ortalama kaçtır?

A) 74 B) 75 C) 76 D) 80

37. -1 bitirme değeriyle ortalama alınırken -1 için ne yapılır?

A) Toplama katılır B) Sayaca katılır C) Çarpıma katılır D) Hesaba katılmaz

38. [18,20,25,19] sıcaklıklarında ≥ 20 olan gün sayısı kaçtır?

A) 1 B) 2 C) 3 D) 4

39. Ortalama = toplam/adet öncesi hangi durum denetlenmeli?

A) adet=0 B) toplam>0 C) indis=0 D) dizi sıralı

40. 0 sayısının onluk yazımında kaç basamak vardır?

A) 0 B) 1 C) 2 D) 10

CEVAP ANAHTARI

Gerekçeli cevaplar

1. **B** - Algoritma bitmelidir; sonsuz döngü bu özelliği bozar.
2. **C** - Karar/koşul için eşkenar dörtgen kullanılır.
3. **D** - $n \times n$ tekrar $O(n^2)$ eder.
4. **B** - Sözde kod algoritmanın dil bağımsız taslağıdır.
5. **A** - İndis biliniyorsa doğrudan erişim sabit zamandır.
6. **C** - string metin dizisini, char tek karakteri temsil eder.
7. **B** - double ondalıklı değerleri tutar.
8. **B** - 17'nin 5'e bölümünden kalan 2'dir.
9. **D** - AND için iki koşul da doğru olmalıdır.
10. **B** - Önce $4+1$ hesaplanır, sonra x'e 5 atanır.
11. **A** - Tek seçimde yanlış koşulun bloğu çalışmaz.
12. **B** - Koşula göre iki daldan tam biri seçilir.
13. **C** - İlk doğru koşul olan ≥ 50 seçilir; sıra önemlidir.
14. **B** - İki sınırın aynı anda sağlanması gerekir.
15. **A** - Bir kararın içinde yeni karar kurulur.
16. **A** - Ön kontrol yüzünden blok hiç çalışmayabilir.
17. **B** - Blok önce, koşul sonra değerlendirilir.
18. **B** - 1,2,3,4,5 olmak üzere 5 tekrar vardır.
19. **B** - Toplamın etkisiz elemanı 0'dır.
20. **D** - Her dış adımda n iç adım vardır: $n \times n$.
21. **B** - Void/prosedür çağırana sonuç değeri döndürmez.
22. **C** - Tanımdaki adlar parametre, çağrıdaki gerçek değerler argümandır.
23. **D** - RETURN dönüş değeri sağlar.
24. **A** - $0! = 1$, bu durum çağrılarının durmasını sağlar.
25. **B** - Çağrılar sonlanmadan sürerek yığını tüketebilir.
26. **C** - İndisler 0 ile $n-1$ arasındadır.
27. **B** - 0 tabanlı üçüncü eleman 6'dır.
28. **A** - Kitaptaki temel dizi modeli aynı tür verileri tutar.
29. **D** - İkinci satırın üçüncü sütunu 6'dır.
30. **C** - Her satırdaki c hücre, r satır için işlenir.
31. **A** - Orta değere göre yarıyı elemek için sıra gereklidir.
32. **C** - Eleman bulunmazsa tüm n öge incelenir.
33. **B** - Kabarcık sıralama komşu çiftleri karşılaştırır.
34. **D** - Hızlı sıralama pivot çevresinde bölümlere ayırır.
35. **C** - Bölme düzeyleri $\log n$, her düzeyde toplam n işlem vardır.
36. **C** - $70 \times 0,4 + 80 \times 0,6 = 28 + 48 = 76$.

37. **D** - Sentinel yalnız döngüyü bitirir.
38. **B** - 20 ve 25 koşulu sağlar.
39. **A** - adet=0 ise sifıra bölme oluşur.
40. **B** - 0 özel durumda tek basamaklıdır.

SON KONTROL

Yanlış yaptığın soruları üniteye göre grupla. Özellikle **koşul sırası, döngü sınırları, indisler, RETURN/YAZ ayrımı, ikili arama ön şartı ve O gösterimi** için birer örneği baştan çöz.